

```

}

int *fun()
{
    static int i = 20;
    return (&i);
}

```

This program just indicates how an integer pointer can be returned from a function. Beyond that, it doesn't serve any useful purpose. This concept can be put to use while handling strings. For example, look at the following program which copies one string into another and returns the pointer to the target string.

```

char *copy (char *, char *);
void main()
{
    char *str;
    char *copy (char *, char *);
    char source[] = "Jaded";
    char target[10];

    str = copy (target, source);
    printf ("ln%s", str);
}

char *copy (char *t, char *s)
{
    char *r;

    r = t;

    while (*s != '\0')
    {
        *t = *s;
        t++;
        s++;
    }
}

```

```

}

*t = '\0';
return (r);
}

```

Here we have sent the base addresses of **source** and **target** strings to **copy()**. In the **copy()** function, the **while** loop copies the characters in the source string into the target string. Since during copying **t** is continuously incremented, before entering into the loop, the initial value of **t** is safely stored in the character pointer **r**. Once copying is over, this character pointer **r** is returned to **main()**.

Functions with Variable Number of Arguments

We have used **printf()** so often without realizing how it works properly irrespective of how many arguments we pass to it. How do we go about writing such routines that can take variable number of arguments? And what have pointers got to do with it? There are three macros available in the file "stdarg.h" called **va_start**, **va_arg** and **va_list** which allow us to handle this situation. These macros provide a method for accessing the arguments of the function when a function takes a fixed number of arguments followed by a variable number of arguments. The fixed number of arguments are accessed in the normal way, whereas the optional arguments are accessed using the macros **va_start** and **va_arg**. Out of these macros, **va_start** is used to initialize a pointer to the beginning of the list of optional arguments. On the other hand, the macro **va_arg** is used to advance the pointer to the next argument.

Let us put these concepts into action using a program. Suppose we wish to write a function **findmax()** which would find out the maximum value from a set of values, irrespective of the number of values passed to it. Here is how we can do it...